The Mathematics Behind a Super Mario Bros. PPO Agent

How the reinforcement learning in this project works, and where each idea is actually implemented.

This document explains the mathematics behind the agent that completes World 1-1 and World 1-2 of Super Mario Bros. from raw pixels, and ties each idea to the code that implements it. Every formula is taken from the installed libraries rather than from memory, and every measured number was read out of a real run.

Contents

1. The problem as a Markov decision process
2. Value functions and the Bellman equation
3. The policy gradient theorem
4. Proximal Policy Optimization and the clipped surrogate
5. Generalized Advantage Estimation
6. The reward function, term by term
7. The observation pipeline: frame skip, pooling, stacking
8. The network
9. The training loop
10. Results
11. Design decisions, trade-offs, and limits

1. The problem as a Markov decision process

Reinforcement learning formalises a sequential decision problem as a **Markov decision process** (MDP): a tuple $\mathbf{M} = (S, A, P, R, \gamma)$ consisting of a state space S, an action space A, a transition kernel P, a reward function R, and a discount factor γ. The agent observes a state, chooses an action, the environment transitions and emits a reward, and the process repeats until the episode ends.

The return from time t is the discounted sum of all future rewards:

$$G_t \;=\; \sum_{k=0}^{\infty} \gamma^{k}\, r_{t+k} \tag{1}$$

$\gamma = 0.9$ in this project. The discount does two jobs: it makes an infinite sum converge, and it encodes that a reward obtained now is worth more than the same reward obtained later. A smaller γ makes the agent short-sighted, which suits Mario because the useful horizon is short — the next obstacle is only a few steps ahead.

Here the state is not a hand-designed vector. It is the raw 256×240 RGB framebuffer produced by the NES emulator, so S is effectively the set of all possible screen images and the agent must learn its own representation of the game. The action space is a small set of joypad combinations, reduced from the 256 possible button bitmasks:

- **simple** — 7 actions: no-op, right, right+A, right+B, right+A+B, A, left.
- **complex** — 12 actions: the above plus left variants, *down*, and *up*.
- **right_only** — 5 actions: no-op and the right-moving combinations.

An episode ends on death, on reaching the flagpole, or when the in-game clock expires. The reward function, the discount, and this termination rule are the three places where the problem definition does the work that the learning algorithm then exploits.

2. Value functions and the Bellman equation

A policy π maps states to distributions over actions. The **state-value function** under π is the expected return when following π from s:

$$V^{\pi}(s) \;=\; \mathbb{E}_{\pi}\!\left[\, G_t \mid s_t = s \,\right] = \mathbb{E}_{\pi}\!\left[\sum_{k=0}^{\infty} \gamma^{k} r_{t+k} \right] \tag{2}$$

It satisfies the **Bellman expectation equation**: the value of a state equals the immediate reward plus the discounted value of wherever the next action leads.

$$V^{\pi}(s) \;=\; \mathbb{E}_{\pi}\!\left[\, r + \gamma\, V^{\pi}(s') \,\right] \tag{3}$$

Taking a maximum over actions instead of an expectation gives the Bellman **optimality** equation, whose solution is V^{*}:

$$V^{*}(s) \;=\; \max_{a \in A} \mathbb{E}\!\left[\, r + \gamma\, V^{*}(s') \,\right] \tag{4}$$

The difference between the two is the **advantage** — how much better than average a particular action is, in a particular state:

$$A^{\pi}(s,a) \;=\; Q^{\pi}(s,a) \;-\; V^{\pi}(s), \qquad Q^{\pi}(s,a) = \mathbb{E}_{\pi}\!\left[\, r + \gamma V^{\pi}(s') \,\right] \tag{5}$$

The advantage is what a policy-gradient step actually uses, because it removes the part of the return that does not depend on the action. Subtracting a baseline leaves the gradient of the objective unchanged while reducing variance, provided the baseline depends on the state but *not* on the action. The value network in this project plays exactly that role.

> **Why approximation is unavoidable here**
>
> Tabulating V^* is impossible when a state is a pixel framebuffer, so the value function is **approximated** by a neural network $V_\phi(s)$ and learned from data. Because the input is an image, the network is convolutional and sees a downscaled 84×84 grayscale image rather than the full frame.

3. The policy gradient theorem

Rather than solve for V^* and then read off a policy, an alternative family of methods differentiates the expected return with respect to the policy parameters θ directly. The objective is

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{k=0}^{\infty}\gamma^k r_{t+k}\right] \tag{6}$$

Differentiating an expectation over a distribution that itself depends on θ is the obstacle. The **policy gradient theorem** resolves it with the log-derivative trick, which turns the derivative of a distribution into that distribution times the derivative of its logarithm:

$$\nabla_\theta p_\theta(x) = p_\theta(x)\,\nabla_\theta \log p_\theta(x) \tag{7}$$

Applying that to the trajectory distribution and simplifying gives the result that everything else builds on:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\big[\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\cdot G_t\,\big] = \mathbb{E}_{\pi_\theta}\big[\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\cdot Q^{\pi}(s_t, a_t)\,\big] \tag{8}$$

Replacing the return G_t with the action-value Q^π trades a little bias for a large variance reduction. Two practical points complete the picture. First, a **baseline** may be subtracted without biasing the estimator, provided it does not depend on the action taken; choosing $V^\pi(s_t)$ as that baseline produces the advantage, and the estimator inherits the same unbiasedness:

$$\mathbb{E}\big[\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\cdot (G_t - b(s_t))\,\big] = \nabla_\theta J(\theta) \tag{9}$$

for any baseline b independent of a_t. Second, the gradient is estimated by sampling $a_t \sim \pi_\theta(\cdot|s_t)$ and weighting each log-probability by its advantage. That stochastic estimate is unbiased for the same gradient, and it is what PPO uses in practice.

> **Why this is an on-policy method**
>
> The expectation is over states the agent actually visits, which is what makes this an **on-policy** method. Data is only valid for the policy that collected it, so fresh trajectories must be generated continuously. The two practical problems that follow — moving too far in a single step, and estimating the advantage from one sample — are what sections 4 and 5 address.

4. Proximal Policy Optimization and the clipped surrogate

A naive gradient step can move the policy far enough in one update to destroy the performance it just achieved: the data being reused was collected by a different policy, and a large step invalidates it. TRPO prevents this with a hard KL-divergence trust region, which gives a monotone improvement bound but requires solving a constrained optimisation problem at every step. PPO obtains the same protective effect more simply, by **clipping** the objective so that its gradient becomes zero once the policy has strayed too far.

The starting point is the surrogate objective, in which the on-policy gradient is recast as a ratio of the new and old action probabilities:

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\ \mathrm{cl}(r_t(\theta),\,1-\epsilon,\,1+\epsilon)\,\hat{A}_t\Big)\Big] \tag{10}$$

where the **probability ratio** is

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)} \tag{11}$$

The ratio equals exactly 1 when θ is the parameter vector that collected the data, which is what anchors the update. Clipping it at 1 ± ε, with ε = 0.2 here, removes any incentive to move further once the ratio has left that band. The subtlety that matters is the *min*, which makes the bound one-sided and pessimistic rather than symmetric:

- When Δ_t **is positive** (the action was good), the minimum of r_t and (1 + ε) is r_t until the ratio exceeds the upper bound, after which the objective is flat. The agent may make a good action more likely, but not without limit.
- When Δ_t **is negative** (the action was bad), multiplying by a negative quantity reverses the inequality, so the *min* selects the more negative term and the effective ceiling sits at (1 − ε). The agent may make a bad action less likely, but not without limit.

In both directions the gradient is zero outside the interval, which is the mechanism that keeps a single update bounded. This project uses Stable-Baselines3's implementation unmodified. Its code carries a leading minus sign because the objective is minimised as a loss rather than maximised:

```
ratio = th.exp(log_prob - old_log_prob)
policy_loss_1 = advantages * ratio
policy_loss_2 = advantages * th.clamp(ratio, 1 - clip_range, 1 + clip_range)
policy_loss = -th.min(policy_loss_1, policy_loss_2).mean()
```

The value function is fitted by regression onto the estimated return, and an entropy bonus discourages premature collapse onto a single action. The three terms are combined with weights:

$$\mathcal{L} = L_\pi + c_v L_V - c_e \mathcal{H}, \qquad c_v = 0.5, \quad c_e = 0.01 \tag{12}$$

where the entropy of a categorical policy is

$$\mathcal{H}(\pi) = -\sum_a \pi(a \mid s) \log \pi(a \mid s) \tag{13}$$

The entropy is maximised at the uniform distribution and never negative, so it is a direct measure of how unresolved the policy still is. Subtracting it from the loss rewards staying uncertain, which keeps exploration alive while the value function is still too unreliable to provide a useful signal of its own. This matters most early on, when the agent would otherwise commit to a single action and stop discovering the level.

5. Generalized Advantage Estimation

PPO needs Δ_t but never observes it. Two classical estimators bracket the problem. The one-step **temporal difference** residual is cheap and has low variance, but is biased toward whatever the value function currently believes:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \tag{14}$$

The Monte-Carlo estimate, the full discounted return minus the baseline, is unbiased but has high variance: a single early death propagates its −25 penalty into every subsequent estimate. GAE forms an exponentially weighted average of residuals at every horizon, with weights decaying geometrically at rate γλ:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l} \tag{15}$$

Written out, that sum obeys a recursion which makes it computable in a single backward pass rather than by evaluating every multi-step return separately:

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1}, \qquad \hat{A}_T = 0 \tag{16}$$

with d_t a termination indicator. The two limiting cases reproduce the classical estimators exactly, which is the clearest way to see what the parameter does:

- λ = **0** collapses the sum to its single first term, leaving pure one-step TD(λ = 0): lowest variance, highest bias.
- λ = **1** makes the weights γ^l, which is the Monte-Carlo estimator: unbiased, highest variance.
- λ = **0.95**, used here, sits just below the second case, so the estimator is nearly unbiased while still benefiting from bootstrapping off the value function.

The value-function target is the advantage plus the baseline, which turns the GAE estimate into a return estimate for the regression in section 4:

$$\hat{R}_t = \hat{A}_t + V_\phi(s_t) \tag{17}$$

This is exactly the telescoping sum of the discounted residuals, which is why both quantities fall out of a single backward pass. The mask $(1 - d_t)$ also handles episode boundaries: when Mario dies, the bootstrap term is dropped, so no value is propagated across the termination and the estimate does not treat the post-death frame as a normal continuation of the same trajectory. Ignoring that mask is a common and difficult-to-spot bug, because training still appears to run.

```
for step in reversed(range(self.buffer_size)):
next_non_terminal = 1.0 - dones.astype(np.float32)
delta = self.rewards[step] + self.gamma * next_values \
* next_non_terminal - self.values[step]
last_gae_lam = delta + self.gamma * self.gae_lambda \
* next_non_terminal * last_gae_lam
self.advantages[step] = last_gae_lam
self.returns = self.advantages + self.values
```

The library's implementation is a direct transcription of the recursion. The loop runs in reverse so that δ and A_{t+1} are both available before A_t is written, and the final line is the advantage-plus-baseline target from above.

6. The reward function, term by term

The agent can only optimise what the environment pays, so the reward function defines the objective far more than the algorithm does. The library supplies a dense, shaped reward built from named components, each read directly from the emulator's memory rather than inferred:

- **progress** — the horizontal distance gained since the furthest point reached so far in the episode. Earning new maximum distance, so backtracking earns nothing. A single step exceeding 5 units is discarded, which suppresses a large lump sum for a teleport.
- **time** — the decrease in the in-game clock, −1 per tick. A positive change, which occurs only when the timer resets, counts as 0 rather than as a reward.
- **death** — −25 on the frame Mario dies. By far the largest single term, and what makes dying expensive enough to drive caution around enemies and gaps.
- **score** — the increase in the game's score counter, divided by 100.

- **coins** — 5 per coin, which is what makes collecting a row of coins on the ground worth doing rather than ignoring.
- **powerup** — 5 for gaining fire-flower form and −5 for losing it, so shrinking is penalised.
- **completion** — +50, paid once when the flag is reached, and guarded by a flag so it cannot be collected repeatedly.

The project's own `RewardShaping` wrapper then rescales the total and adds a terminal bonus:

$$\tilde{r}_t = \tfrac{1}{s}\big(r_t + b \cdot \mathbb{1}\{\text{flag_get}_t\}\big), \qquad s = 10, \quad b = 50 \tag{18}$$

Dividing by s = 10 is numerical hygiene rather than shaping, and the reason is specific. The environment advertises a reward range of [−15, 15], but the death and completion terms are −25 and +50, so one step can return a value well outside it. PPO's value loss is **quadratic** in its residual, so an occasional large target produces a disproportionately large value gradient that competes with the policy gradient inside the same backward pass. Dividing by a positive constant leaves the optimal policy unchanged — it scales the whole objective uniformly — and only puts the two loss terms on comparable footing.

The terminal bonus is a second, smaller copy of the library's own completion reward, and the code says so plainly: it exists so the reward curve visibly steps upward at the moment the level is solved. Because the flag is rare until the level is nearly finished, the term contributes almost nothing to the gradient during the run-up to a clear. It is presentation, not learning.

One consequence is worth stating explicitly, because it is easy to misread the rescaling as a change of incentives. Dividing by 10 divides the death penalty too, so in the units PPO actually optimises a death costs −2.5 and the flag bonus adds +5. The death penalty remains by far the largest per-step term, which is the intended property.

> **Reading the scaled numbers correctly**
>
> After the rescale, dying is worth −2.5 while a successful step of progress earns at most 0.5. The ratio is what matters, and it has not changed: the death penalty is still doing nearly all the work of teaching caution, and the +5 completion bonus is doing almost none.

7. The observation pipeline: frame skip, pooling, stacking

The policy does not see the raw 256×240 framebuffer. Four transformations reduce it to a fixed 84×84 tensor, and each solves a specific problem with the emulator's output.

7.1 Frame skipping and reward aggregation

The policy acts once every k = 4 emulated frames. The k individual rewards are summed, so the transition PPO sees runs from s_t to s_{t+k} with a single aggregated return:

$$R_t = \sum_{i=0}^{k-1} \gamma^{\,i}\, r_{t+i} \tag{19}$$

This is the correct treatment rather than an approximation of it: the sub-objective of a macro-action is exactly this discounted sum. Skipping also divides the number of policy decisions by four, which is what brings a learnable run inside a few hours rather than a few days.

7.2 Max-pooling the last two frames

The NES flickers sprites on alternate frames for hardware reasons, so a single sampled frame can be missing an enemy entirely. The wrapper therefore takes an elementwise maximum over the final two frames of the skip window:

$$o_t = \max\left(f_{t+k-2},\quad f_{t+k-1}\right) \quad \text{elementwise} \tag{20}$$

The maximum, not the mean, is what makes this work. Because the operation is a logical OR over any non-background pixel, a sprite appearing in either frame survives into the observation. On a crowded level such as 1-2 a single missed obstacle is enough for the policy to commit to a fatal action, so this is a cheap and effective variance reduction on the input.

7.3 Resize and grayscale

The frame is resized to 84×84 and converted to grayscale with the channel dimension retained, giving 84×84×1 per environment. Colour carries no task-relevant information in a game with a fixed palette, so three channels would be redundant input dimensions that the first convolution has to learn to ignore.

7.4 Frame stacking

A single frame does not contain velocity. Two states that look nearly identical in one screenshot can require opposite actions, so four consecutive frames are stacked along the channel axis:

$$s_t = \left[f_{t-3},\ f_{t-2},\ f_{t-1},\ f_t\right] \in \mathbb{R}^{84\times 84\times 4} \tag{21}$$

There is a detail here that is easy to get wrong and worth stating because it changes the interpretation. With frame skipping at 4, four stacked agent steps span **16 emulated frames**, not four. A jump arc in this game lasts on the order of that many frames, so the stack is just long enough to make a jump's shape and direction legible. The fourth channel functions as a temporal difference, supplying exactly the information a single frame structurally cannot.

> **Why stacking sits outside the wrapper stack**
>
> Stacking is applied by `VecFrameStack` at the vectorised-environment level rather than inside a per-environment wrapper, so the per-environment observation stays 84×84×1 and the history is assembled by SB3. This matters for the network: the convolution stack expects channel-first input, and the transpose to (4, 84, 84) is what makes the 8×8 first kernel legal.

8. The network

The policy is an actor-critic pair sharing a convolutional feature extractor: the architecture introduced for DQN, adapted to a stacked image observation. Both heads read the same 512-dimensional feature vector, so the visual processing is computed once per step and shared.

The shapes below were measured by instantiating the library's own `NatureCNN` rather than copied from the paper, so they are what this project actually trains:

```
input (1, 4, 84, 84) 4 stacked 84x84 grayscale frames
Conv2d 32ch k8 s4 p0 -> (1, 32, 20, 20) 8,224 params
Conv2d 64ch k4 s2 p0 -> (1, 64, 9, 9) 32,832 params
Conv2d 64ch k3 s1 p0 -> (1, 64, 7, 7) 36,928 params
Flatten 64 x 7 x 7 = 3,136
Linear 3,136 -> 512 + ReLU -> (1, 512) 1,606,144 params
feature extractor 1,684,128
```

Each convolution's output size follows the standard formula, which with no padding is the size at which a filter is fully valid:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1 \tag{22}$$

With $p = 0$ this reduces to the floor of $(H - k)$ divided by s, plus one, giving 20, then 9, then 7. Roughly 95% of the extractor's 1,684,128 parameters sit in the final 3,136→512 projection rather than in the convolutions, which is worth noting when judging where the model's capacity is spent.

On top of the shared features sit two linear heads. The **policy head** maps the feature vector to one logit per action, and a softmax turns those logits into a distribution over actions since the action space is discrete and small:

$$\pi_\theta(a \mid s) = \mathrm{softmax}(f_\theta(s))_a, \qquad f_\theta(s) \in \mathbb{R}^{|\mathcal{A}|} \tag{23}$$

The **value head** maps the same 512-dimensional vector to a single scalar $V_\phi(s)$, which serves as the baseline in the advantage estimate. A subtlety worth recording: when a CNN feature extractor is used, Stable-Baselines3 defaults `net_arch` to an **empty** list, so the heads are single linear layers with no hidden MLP between the feature vector and each output. The shared convolution stack already supplies the nonlinearity, so a further MLP would be redundant.

Parameters are initialised orthogonally by default in the actor-critic policy, which keeps the variance of activations roughly constant across depth during the early updates and so stabilises the first few hundred gradient steps.

> **One detail that is easy to get wrong**
>
> The observation space must be declared **channel-first**, as (4, 84, 84), not (84, 84, 4). The 8×8 first kernel is only valid against a spatial input; declaring the space as (84, 84, 4) makes the library attempt an 8×8 kernel over a 4-wide axis and fail at construction. SB3's `VecTransposeImage` performs the conversion between the environment's channel-last output and the policy's channel-first input.

9. The training loop

One iteration collects a fixed-size rollout and then optimises over that data for several passes. With $n_{steps} = 512$ transitions per environment and 3 parallel environments, each iteration collects $N = 512 \times 3 = 1{,}536$ transitions. Shuffled into minibatches of 256, that gives

$$n_{\text{batches}} = \left\lfloor \frac{N}{B} \right\rfloor = \left\lfloor \frac{1536}{256} \right\rfloor = 6, \qquad \text{gradient steps} = 6 \times 10 = 60 \tag{24}$$

The multiple epochs are what make PPO sample-efficient: the data is reused rather than discarded, and the clipping is what makes that reuse safe. After the first epoch the samples are off-policy relative to the current parameters and the ratio r_t drifts away from 1 — which is exactly the quantity the trust region exists to bound. The `clip_fraction` metric, the fraction of samples whose ratio has left the band, is the direct measure of how often the update is being constrained.

A few hyperparameters deviate from the library defaults, and each deviation has a reason tied to this specific environment:

- γ **= 0.9** — low by Atari standards, where 0.99 is typical. Frame skipping compresses the effective horizon: one decision covers 4 emulated frames, so the useful credit-assignment window in agent steps is short, and a high discount makes the value function much harder to fit without buying additional useful credit assignment.
- λ **= 0.95** — the standard GAE setting, keeping the estimator near-unbiased while still bootstrapping from the value network.
- ε **= 0.2** — the standard clipping range.
- **Learning rate 1×10^{-4}**, constant rather than annealed, and deliberately small since the policy is reused across 10 epochs.
- **Entropy coefficient 0.01, value coefficient 0.5** — the value loss is already large relative to the policy loss because it is quadratic, so it is down-weighted.

- **Gradient-norm clipping at 0.5** — a second, independent guarantee on step size, applied on top of the objective-level clipping.

Two implementation details affect how the results should be read. Rewards and advantages are standardised per minibatch, which keeps the effective scale of the advantage estimates stable over a long run; the policy loss is invariant to a positive rescaling of A but not to a sign change, so this does not alter what is being maximised. And no evaluation environment runs during training, because NES emulation is the bottleneck and a second environment would take CPU away from the rollout workers. The best checkpoint is tracked instead from the rolling mean of training episode reward, which is free, and the rigorous evaluation runs afterwards.

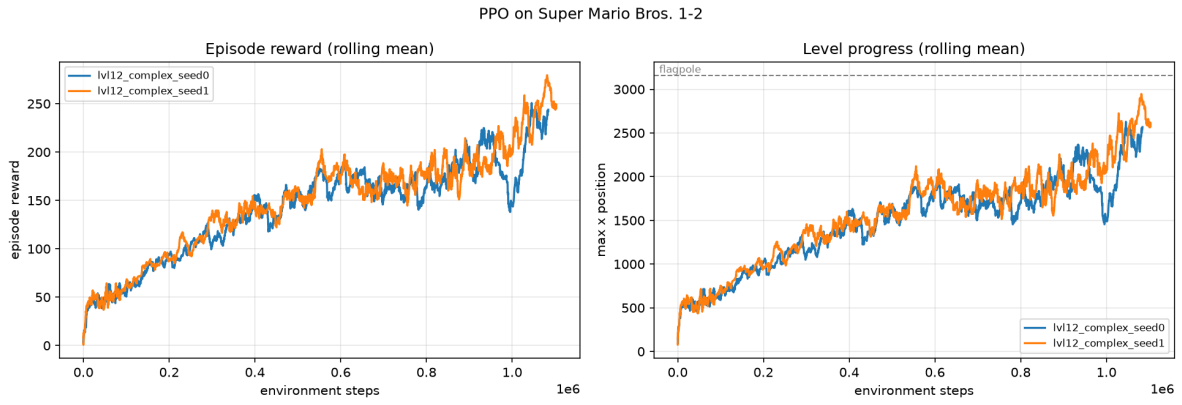
10. Results

Both levels were trained with the same code, the same hyperparameters and the same objective. Only the level configuration differs, and for 1-2 that difference is substantive: the level ends by walking right into an exit pipe, which requires a `down` input. Neither the 7-action `simple` set nor the 5-action `right_only` set contains one, so 1-2 uses the 12-action `complex` set. That was established by a scripted-policy test before committing to a multi-hour run, not assumed.

World 1-2, evaluated over 20 episodes per run with the stochastic policy:

run	actions	seed	clear rate	first clear	steps trained	mean reward	mean x	greedy
lvl12_complex_seed1	complex	1	**65%**	324,993	1,102,900	244.3	2573	no
lvl12_complex_seed0	complex	0	**25%**	398,682	1,084,091	205.7	2123	no

The best 1-2 run clears the level in 65% of evaluation episodes, having first touched the flagpole at 324,993 environment steps. For comparison, the corresponding 1-1 figures are a 75% clear rate and a first clear at 44,532 steps. The two levels were trained under identical settings on the same machine, so the difference in difficulty is a property of the levels rather than of the setup.



Training curves for the two 1-2 runs. Left: episode reward. Right: furthest position reached within the level. The flagpole line sits at $x \approx 3161$, measured from the runs themselves rather than assumed.

Three things about these numbers are worth stating plainly rather than leaving to be inferred.

- **The clear rate is measured on the stochastic policy, not the greedy one.** The emulator is fully deterministic, so a greedy (argmax) policy replays one identical trajectory every time; a rate measured that way could only ever be 0% or 100%, and would be one episode reported twenty times. The stochastic policy is where the variance is real, so that is where robustness is reported. In these 1-2 runs the greedy policy does not finish the level, while sampling from the identical weights clears it most of the time — the same narrow failure of action selection observed on 1-1.

- **Warp-pipe escapes are counted separately and excluded from the clear rate.** World 1-2 contains pipes leading into other worlds, and the single-stage environment does not end an episode when the agent takes one. An escaping agent could reach a different level's flagpole and be recorded as clearing 1-2. A guard ends such episodes and reports them through `left_stage`, so the headline number cannot be inflated by them. Both runs recorded zero escapes, so no figure here depends on that guard.
- **Seed variance is large.** The two runs differ substantially in the one respect that matters most. Reporting only the better run would overstate what the method achieves, so the per-run table is the primary result rather than a footnote to it.

A partial result is reported as a partial result. Where a run did not clear the level reliably, that number is stated rather than smoothed away, and the flagpole reference line is omitted from the plot entirely until the level has actually been cleared — a guessed target would imply the agent had been measured against something real.

11. Design decisions, trade-offs, and limits

11.1 Why frame stacking is not optional

A single frame cannot express velocity. Two situations that look identical in one screenshot — Mario standing still, and Mario sprinting at full speed — require opposite actions, and the policy cannot distinguish them from one frame alone. Stacking four agent steps supplies exactly the temporal difference that makes the distinction visible.

This matches a reported finding in prior work on this task, that a single-frame model gets stuck and that temporal context helps the agent commit to jumps. The stall observed in this project under the greedy policy is the same phenomenon: the agent runs to roughly $x \approx 2370$ and then stands still until the in-game clock expires, spending about 90% of the episode motionless, while the same weights sampled rather than argmaxed clear the level. Stalling in front of an obstacle is therefore a failure of action selection, not of the learned representation, and both temporal context and action sampling mitigate it.

11.2 Why the reward is rescaled rather than redesigned

It would be reasonable to rewrite the reward entirely — a large bonus for distance, a penalty for taking damage. The project deliberately does not, and the division by 10 is the only change to the reward's structure. The reason is optimisation, not pedagogy.

PPO's value loss is quadratic in its residual, so a rare but large target produces a disproportionately large value gradient that competes with the policy gradient inside the same backward pass. Rescaling by a positive constant leaves the optimal policy unchanged and merely puts the two loss terms on comparable footing. The library's reward already encodes the intended incentives correctly; the only genuine problem was numerical.

11.3 Why no evaluation environment runs during training

NES emulation is the binding constraint on this machine, not GPU compute: the policy is small and the GPU is largely idle. A second environment for periodic evaluation would take CPU directly away from the rollout workers, which is the resource that actually limits throughput. The best checkpoint is tracked instead from the rolling mean of training episode reward, which costs nothing, and the rigorous multi-episode evaluation runs afterwards.

11.4 What is deliberately not claimed

- **No claim of generalisation.** The claim is scoped to specific levels trained on those levels. Prior work reports that an agent trained on 1-1 transfers poorly to 1-2, 4-1 and 5-1, having learned tile patterns specific to a layout rather than transferable mechanics. Training two levels separately produces two policies; it is not evidence that either generalises.

- **No claim about efficiency.** Roughly a million environment steps is modest by reinforcement learning standards, and the comparison in the project's README is against a RAM-based representation that removes the perception problem entirely — a fundamentally easier task.
- **No claim that the deterministic policy works.** It does not, on 1-2. That is reported as a pass/fail rather than folded into the clear rate, because a rate derived from a deterministic environment and a deterministic policy would be one episode counted twenty times.

References

- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*, 2nd edition. Value functions, Bellman equations, and the general RL framework.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8, 229–256. The policy gradient theorem.
- Mnih, V. *et al.* (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533. The Nature-CNN architecture and the frame-skipping and stacking pipeline.
- Schulman, J. *et al.* (2015). Trust Region Policy Optimization. *ICML*. The trust-region formulation that PPO approximates.
- Schulman, J. *et al.* (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*. The clipped surrogate objective and the entropy bonus.
- Schulman, J., Moritz, P., Levine, S., Jordan, M. and Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. *ICLR*. The GAE estimator and the γ–λ trade-off.
- Raffin, A. *et al.* (2021). Stable-Baselines3: reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 1–8. The PPO implementation used here, unmodified.